DG1IAD, Internet Applications & Database Design

Aston University
BIRMINGHAM UK

Dr Alexandros Giagkos

Computer Science
College of Engineering and Physical Sciences

Database Development

- Week 1: Relational Databases and SQL
  - Lab: Install XAMPP, and perform CRUD operations
- Week 2: Functional Dependences and Normal Forms
  - Lab: Practising relationships and normalised designs
- **Week 3: Data Integrity, ACID and Transactions**
  - **Lab: Complete unfinished lab work**

Data Integrity, ACID and Transactions

Outline:

- Part I: Data integrity and constraints
- Part II: The ACID properties and Transcations

Learning outcomes:

- Learn how to ensure data integrity in relational databases
- Understand the concept of transactions

Motivation of today's topics

- **Healthy data**: We want all stored information to be *valid* and *true* for our users.
- **Loss Updates**: Different users want to make several changes to the same record of data at the same time
- **Durability and Recovery**: We expect the RDBMS to be able to recover from any fault/disaster/etc.

Part I

Data integrity and constraints

Integrity, why and how?

- *(Logical)* Data integrity is lost when things do not update in a consistent manner (remember data anomalies!)
- Human errors or malicious acts
- It is ensured by applying various **constraints**
- Constraints relate to *entities*, *domain* or *relationships* and can also be *user-defined*
- The majority of integrity constraints are addressed in DDL

Entity Integrity

- Entity integrity relies on the use of primary keys
- Each tuple should be uniquely identified, and contain data related to that entity (aka no repeated data)
- In SQL, we use `PRIMARY KEY`

```sql
CREATE TABLE department (
        dept_name VARCHAR(20),
        building VARCHAR(150),
        budget NUMBER(12,2),
        PRIMARY KEY (dept_name)
);
```

Entity Integrity with an example

lecturer (lec_id, name, .., city, stu_id)

lec_id	name	..	city	stu_id
L1	Alex	..	B'ham	NULL
NULL	Anne	..	..	S1

- Note that foreign keys may have NULL values (for partial participation)

From Oracle's online documentation:

A primary key defines the set of columns that uniquely identifies rows in a table.

When you create a primary key constraint, none of the columns included in the primary key can have NULL constraints; that is, they must not permit NULL values.

A table can have at most one PRIMARY KEY constraint.

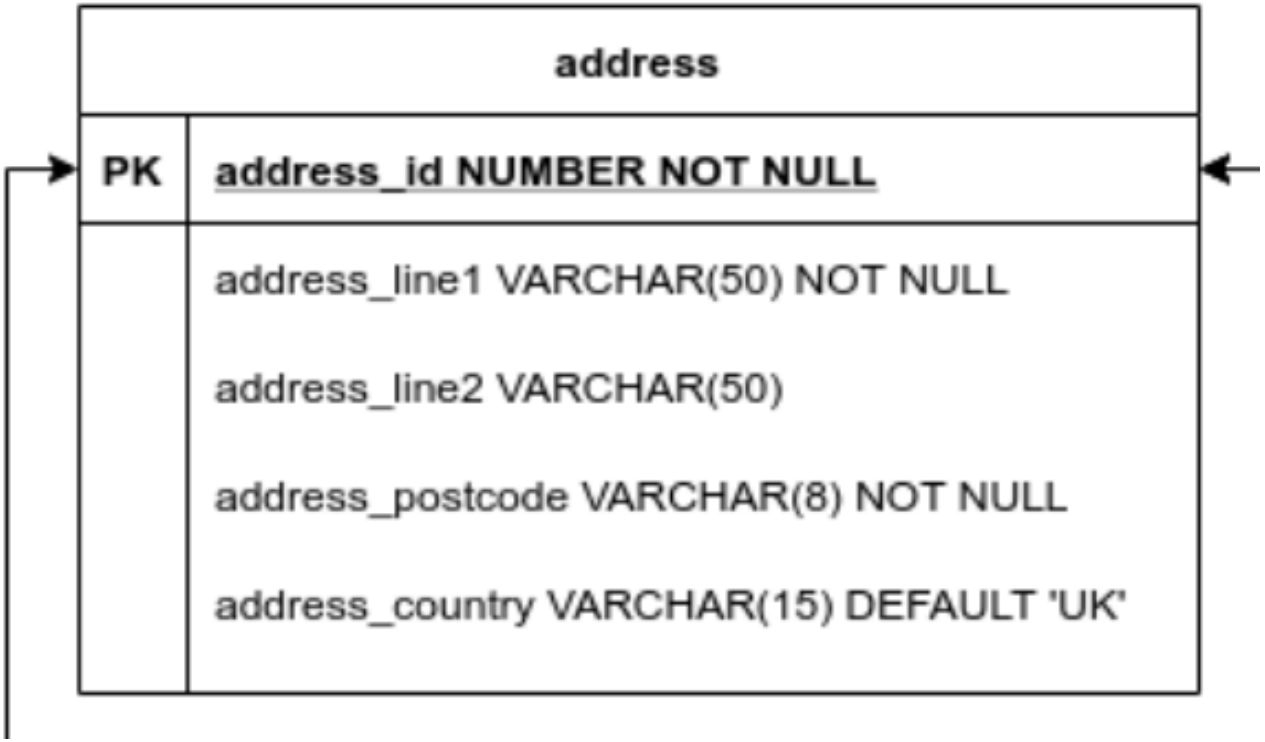
Domain Integrity

- Domain integrity lists all allowable values for an attribute
- VARCHAR of 100 characters, DECIMAL of 4 digits and 2 decimal points, etc.

```sql
CREATE TABLE course (
  course_id      VARCHAR(7) ,
  title          VARCHAR(50) NOT NULL,
  dept_name      VARCHAR(20) NOT NULL,
  credits        DECIMAL(2,1) DEFAULT 1.5
  PRIMARY KEY (course_id)
);
```

Domain Integrity in a detailed E-R diagram

- Domain integrity constraints are detailed in E-R diagrams



Referential Integrity

- Also called foreign-key constraints, ensure that data is stored and used uniformly
- In SQL, we use `FOREIGN KEY`, `REFERENCES`

```sql
CREATE TABLE course (
  course_id      VARCHAR(7) ,
  title          VARCHAR(50) NOT NULL,
  dept_name      VARCHAR(20) NOT NULL,
  PRIMARY KEY (course_id),
  FOREIGN KEY (dept_name)
        REFERENCES department(dept_name)
);
```

- Note that `NOT NULL` for dept_name ensures *full participation*

User-defined Constraints

- User-defined constraints are business rules that make sense in the context of the application, e.g.:
  - *A teacher cannot teach more than 3 modules per term*
  - *Any course's number of credits cannot exceed 3.0*

```
CREATE TABLE course (
        course_id        VARCHAR(7) ,
        title            VARCHAR(50) NOT NULL,
        dept_name        VARCHAR(20) NOT NULL,
        credits DECIMAL(2,1) DEFAULT 1.5
                         CHECK(credits <= 3.0),
        PRIMARY KEY (course_id)
);
```

- `CHECK` can be used to enforce such constraints

Discussion

- Normal forms ensure "good design", i.e., correct schema decomposition
- Integrity constraints ensure correct and consistent data
- They are interrelated and equally important

Part II

The ACID properties and Transcations

Definition of a Transaction

- A transaction is a collection of several instructions that form a single unit of work
- In practice, it is the execution of a sequence of SQL queries on the database that perform a high-level function
- The RDBMS must ensure that either the entire transaction is executed or none of its instructions

Initiating a Transaction

- Transactions are initiated by the end-users using:
  - A high-level data-manipulation language (i.e., SQL)
  - A programming language with embedded database access capabilities (ODBC, JDBC)
- They are delimited by statements of the form begin transaction and end transaction
- They are indivisible; if one statement fails, any changes to the database must be undone

  Statement failures (e.g., divide by 0), Operating system failures (e.g., memory loss), Hardware issues (e.g., I/O errors)

Transactions in Oracle

- DDL and DCL commands are auto-committed
- DML commands need to be managed by the end-user
- A transaction begins with the first DML command
- A transaction ends when
  - `COMMIT` or `ROLLBACK` are used
  - A DDL or DCL command is used (thus has to auto-commit)
  - Command editor exits normally
  - System crashes
- `SAVEPOINT` can be used as a marker for rolling back
  - `ROLLBACK TO` <name_of_savepoint>
- Auto-commit can be turned on and off

Transactions in Oracle - An Example

- An example of a transaction

```
CREATE TABLE People (
  fname VARCHAR2(15),
  lname VARCHAR2(15),
  height NUMBER(3,2),
  weight NUMBER(5,2)
);
// transactions starts here
SAVEPOINT tbl_creation;

INSERT INTO People VALUES ('Jenny', 'Evans', 1.67, 54);
SAVEPOINT data_insertion;

SELECT * FROM People;

ROLLBACK TO tbl_creation;

SELECT * FROM People;
```

FNAME	LNAME	HEIGHT	WEIGHT
Jenny	Evans	1.67	54

no data found

Important Transaction Properties

- A transaction in a DBMS must maintain the ACID properties:
  - **Atomicity:** "all-or-none", a transaction must be treated as an atomic unit and must leave the database in a healthy state
  - **Consistency:** "all seem correct", the state of the database after the transaction must be healthy, and data must be consistent
  - **Isolation:** "as if alone", all statements of a transaction must execute no matter how many other transactions or single queries are performed *in parallel*
  - **Durability:** "prone to failures", the database must be able to return to a healthy state after a crash or failure, completing any affected transactions

A practical example

- Scenario: "Transferring money from account A to account B in a very simple bank application"
- We will assume only read, write and simple arithmetic operations are possible
- Consider the following transaction

```
T: read(A);
   A := A - 100;
   write(A);
   read(B);
   B := B + 100;
   write(B)
```

A practical example: Atomicity

```
1  T: read(A);
2     A := A - 100;
3     write(A);
4     read(B);
5     B := B + 100;
6     write(B)
```

- If the database crashes after line 3, the resulting state will not reflect the state of the world, it will be an inconsistent state
- Inconsistent states occur during transactions but are replaced by consistent states
- The recovery system of the DBMS is able to keep track of the values that are changed, so it can recover if needed

A practical example: Consistency

```
T: read(A);
   A := A - 100;
   write(A);
   read(B);
   B := B + 100;
   write(B)
```

- **Data consistency:** automatic testing for data and referential integrity constraints
- **Transaction consistency:** high-level constraints thus the responsibility of the application programmer
- The consistency requirement is that the sum of A and B does not change after the transaction

A practical example: Isolation

```
1  T: read(A);
2     A := A - 100;
3     write(A);
4     read(B);
5     B := B + 100;
6     write(B)
```

- The state of the database is temporarily inconsistent between lines 3-6
- Note that executing transactions serially may solve this problem, but concurrent transactions provide significant performance benefits

A practical example: Durability

```
1  T: read(A);
2     A := A - 100;
3     write(A);
4     read(B);
5     B := B + 100;
6     write(B)
```

- If the transaction completes successfully after line 6, any changes to the database must persist
- Failures may cause memory loss (thus some instructions may fail), but a successful transaction must imply that changes are written on disk
- The recovery system is able to reconstruct transaction changes, when the system is restarted after a failure

Ensuring Transaction Atomicity: Shadow Paging

- Shadow paging, conceived by IBM, was a very popular approach in the 80s
  - The DBMS makes a copy of the *pages* a transaction is about to change
  - If the transaction is successful, the copy becomes the original (and visible to other users)
  - Too slow and problematic
  - Today only a few DBMS use it still (e.g., CouchDB, OpenLDAP's database backend)

Ensuring Transaction Atomicity: Logging

- Logging is the most common approach
  - The DBMS records all actions so it can undo them if the transaction is aborted
  - Undone records are maintained in both volatile and non-volatile storage (similar to a blackbox of an airplane)
- Logging at that level is used by almost everyday DBMS
- In addition to rolling back in atomicity, it has more benefits:
  - Auditing
  - Efficiency reasons

Ensuring Transaction Consistency

- Remember that we try to make sure that the following statement holds:

  "If what is represented by the database is logically correct, then any SQL queries sent to this database will produce logical results sets"

- We ensure data consistency by enforcing domain integrity and primary-foreign key constraints

- We rely upon the application programmer for transaction consistency

Ensuring Transaction Isolation

- From the end-user's point, there needs to be an exclusive access to the database
  - A transaction is submitted, and is executed as if it is running by itself
- But concurrency is achieved by interleaving the transactions because running transactions in a FIFO order is not efficient

Concurrency Example

- In the bank account scenario, A and B each have £600
  - T_1: transfer £50 from A to B
  - T_2: credit both A and B with 6% interest

T_1	T_2
BEGIN	**BEGIN**
read(A)	read(A)
A:= A-50	A:= A*1.06
write(A)	write(A)
read(B)	read(B)
B:= B+50	B:= B*1.06
write(B)	write(B)
COMMIT	**COMMIT**

- How many possible outcomes can result from T_1 and T_2?

Concurrency Example (cont.)

- There is no guarantee that either T_1 or T_2 will execute before the other, if they are submitted together
- But the net effect should be equivalent, i.e., the consistency constraint A + B = 1272 must hold no matter the order
  - A = 583, B = 689, A + B = 1272
  - A = 586, B = 686, A + B = 1272
- The outcome depends on whether T_1 executes before T_2 and vice versa

Concurrency using Serial Schedules

- Two equivalent serial schedules are possible for T_1 and T_2:

T_1	T_2
BEGIN	
read(A)	
A:= A-50	
write(A)	
read(B)	
B:= B+50	
write(B)	
COMMIT	
	BEGIN
	read(A)
	A:= A*1.06
	write(A)
	read(B)
	B:= B*1.06
	write(B)
	COMMIT
A=583, B=689	

$\equiv$

T_1	T_2
	BEGIN
	read(A)
	A:= A*1.06
	write(A)
	read(B)
	B:= B*1.06
	write(B)
	COMMIT
BEGIN	
read(A)	
A:= A-50	
write(A)	
read(B)	
B:= B+50	
write(B)	
COMMIT	
A=586, B=686	

Concurrency using Interleaving

- Serial schedules are slow due to disk and network I/O, breakpoints, waiting for user input, etc.
- Interleaving: when one transactions stalls, another continues

T_1	T_2
BEGIN	
read(A)	
A:= A-50	
write(A)	
	BEGIN
	read(A)
	A:= A*1.06
	write(A)
read(B)	
B:= B+50	
write(B)	
COMMIT	
	B:= B*1.06
	write(B)
	COMMIT

$\equiv$

Either of the serial schedules as long as instructions that affect similar tuples are not mixed

Bad Schedules due to Interleaving

- Interleaving can cause atomicity and consistency issues

T_1	T_2
BEGIN	
read(A)	
A:= A-50	
write(A)	
	BEGIN
	read(A)
	A:= A*1.06
	write(A)
	read(B)
	B:= B*1.06
	write(B)
	COMMIT
read(B)	
B:= B+50	
write(B)	
COMMIT	

$\not\equiv$

Either of the serial schedules
as A = 583, B = 686
thus A+B = 1269 $\neq$ 1272
(bank missing £3)

Discussion on Interleaving

- Interleaving is a neat method to allow transcations happen
- It can only work if the transactions can be considered conflict serialisable
- **Otherwise** interleaving is prone to **conflicts**!

  Conflicts happen when:
  - → Instructions that belong to different transactions ..
  - → .. affect the same object (tuple), and at least one of them is a write (R-W, W-R, W-W)

Example of R-W Conflict

- Schedule leading to *unrepeatable reads* (different values) due to atomicity and consistency issues
- Assume that the initial value of A = £600

T_1	T_2
BEGIN	
read(A)	
	BEGIN
	read(A)
	A:= 700
	write(A)
	COMMIT
read(A)	
COMMIT	

- If a consecutive read of T_1 returns new rows introduced by T_2, the conflict is called *phantom reads*

Example of W-R Conflict

- Schedule leading to reading uncommitted data (aka *causes dirty reads*)

T_1	T_2
BEGIN	
read(A)	
A:= 800	
write(A)	
	BEGIN
	read(A)
	A:= A - 50
	write(A)
	COMMIT
ABORT	

- T_1 has to abort and rollback to a consistent state
- T_2 assumed A = 800 (uncommitted)

Example of W-W Conflict

- Schedule leading to *overwriting uncommitted data*

T_1	T_2
BEGIN	
A:= 500	
write(A)	
	BEGIN
	A:= 700
	write(A)
	B:= "To Jenny"
	write(B)
	COMMIT
B:= "To Mark"	
write(B)	
COMMIT	

- Both T_1 and T_2 make changes to A and B
- A and B should be modified atomically

Solutions available in DBMS's

- Concurrency Control
  - The DBMS automatically inserts lock/unlock requests and schedules actions for different transactions
  - It ensures that the resulting transactions' execution is equivalent to executing them one after the other in some order
  - Protocols such as the Two-Phase Locking are in place

    (See Chapters 17 and 18, Silberschatz, A., Korth, FH., and Sudarshan, S., Database System Concepts, McGraw-Hill, 2020)

High-level illustration of using Locks

- The protocol implements a Lock Manager component that works as a referee

T_1	T_2	Lock Manager
BEGIN		
LOCK(A)		Grant ($T_1 \rightarrow A$)
read(A)	**BEGIN**	
	LOCK(A)	Deny
write(A)		
read(A)		
UNLOCK(A)		Release ($T_1 \rightarrow A$)
	read(A)	Grant ($T_2 \rightarrow A$)
	write(A)	
COMMIT	UNLOCK(A)	Release ($T_2 \rightarrow A$)
	COMMIT	

- On cascading aborts and deadlocks, the DBMS stops transactions that cause troubles

Ensuring ACID properties in real life

- DBMS implement various ways to ensure all ACID properties
- Isolation levels allow the Database Administrator to define the degree to which a transaction much be isolated

Isolation Level	Dirty reads	Non-repeatable reads	Phantoms
Read Uncommitted	May occur	May occur	May occur
Read Committed	Don't occur	May occur	May occur
Repeatable Read	Don't occur	Don't occur	May occur
Serializable	Don't occur	Don't occur	Don't occur

- Data-driven applications are restricted when ACID is fully enforced
  - When is it OK to allow conflicts? When isn't?

Summary

You can now better discuss the following topics:

- Data integrity and constraints that ensure it in relational databases
- Transactions and their ACID properties